SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1707**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

Code of Conduct:

*I **S. Lakshmi Priya (192524284)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *S. Lakshmi Priya*

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

Answer:

Propositional symbols:

Symbol	**Meaning**
F	Patient has Fever
C	Patient has Cough
R	Patient has Rash
B	Patient has Breathlessness
Flu	Possible Flu
Mea	Possible Measles
Pne	Possible Pneumonia

Rules (knowledge base):

R1: $F \land C \rightarrow Flu$

R2: $F \land R \rightarrow Mea$

R3: $C \land B \rightarrow Pne$

Facts about Patient A: F = True, C = True, B = True (R is not known, so it is not asserted).

KB = { $F \land C \rightarrow Flu$, $F \land R \rightarrow Mea$, $C \land B \rightarrow Pne$, F, C, B }

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

Answer:

Known facts at the start: {F, C, B}. Rules are checked repeatedly until no new fact is added.

Step	**Rule checked**	**Premises satisfied?**	**Action / Fact derived**
1	R1: $F \land C \rightarrow Flu$	F = T, C = T → Yes	Fire R1; derive Flu
2	R2: $F \land R \rightarrow Mea$	F = T, R unknown → No	Cannot fire
3	R3: $C \land B \rightarrow Pne$	C = T, B = T → Yes	Fire R3; derive Pne
4	R1, R2, R3 re-checked	No new premises true	No new facts – fixed point reached

Final fact set = {F, C, B, Flu, Pne}.

Diagnoses for Patient A: Possible Flu and Possible Pneumonia. Measles is not derived because Rash is not a known fact.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Answer:

Goal: Pne (Patient A has Pneumonia). Backward chaining starts from the goal and finds a rule whose conclusion matches it.

1. Goal Pne. The only rule with Pne as its conclusion is R3: C ∧ B → Pne. Replace the goal by the sub-goals C and B.
2. Sub-goal C: it is a known fact (Cough = True) → proved.
3. Sub-goal B: it is a known fact (Breathlessness = True) → proved.
4. Both sub-goals are proved, so the goal Pne is proved.

Goal reduction (AND) tree:

Pne (goal)
|
R3: C ∧ B → Pne (AND node)
/ \
C (fact ✓) B (fact ✓)

Conclusion: the goal 'Patient A has Pneumonia' is verified (TRUE). Only 2 facts and 1 rule were needed; rules R1 and R2 were never explored, which shows that backward chaining is goal-directed and more efficient than forward chaining here.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

Answer:

Rule 1: ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x). Its premises are Vehicle(x) and EmergencyType(x). In clause form: ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x).

Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance).

Step	Pair to unify	Substitution θ	Result
1	Vehicle(x) with Vehicle(Ambulance)	θ1 = {x / Ambulance}	Unified; x bound to Ambulance
2	EmergencyType(x)θ1 = EmergencyType(Ambulance) with fact EmergencyType(Ambulance)	θ2 = { } (identity)	Identical, no new binding
3	Compose substitutions	θ = θ1 ∘ θ2 = {x / Ambulance}	MGU
4	Apply θ to the conclusion ClearPath(x)	ClearPath(x)θ	ClearPath(Ambulance)

Alternative attempt: Vehicle(x) with Vehicle(CarA) gives {x / CarA}, but EmergencyType(CarA) is not a fact (and ≠ EmergencyType(Ambulance)), so this binding fails.

Complete MGU: θ = {x / Ambulance}. Derived fact: ClearPath(Ambulance).

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

Answer:

Goal: Proceed(CarA). Variables are standardised apart and the quantifiers are dropped after Skolemisation (all are universal, so no Skolem terms are needed).

Step 1 – Convert to CNF:

C1: ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x) (Rule 1)

C2: ¬ClearPath(x) ∨ GreenSignal(x) (Rule 2, part 1: ¬A ∨ (B ∧ ¬C) distributes into two clauses)

C3: ¬ClearPath(x) ∨ ¬RedSignal(x) (Rule 2, part 2)

C4: ¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x) (Rule 3)

C5: Vehicle(Ambulance)

C6: EmergencyType(Ambulance)

C7: Vehicle(CarA)

C8: Behind(CarA, Ambulance)

C9: ¬Proceed(CarA) (negated goal)

Step 2 – Resolution steps:

No.	Resolve	Unifier	Resolvent
R1	C9 with C4	{x / CarA}	¬Vehicle(CarA) ∨ ¬Behind(CarA,y) ∨ ¬GreenSignal(y)
R2	R1 with C7	{ }	¬Behind(CarA,y) ∨ ¬GreenSignal(y)
R3	R2 with C8	{y / Ambulance}	¬GreenSignal(Ambulance)
R4	R3 with C2	{x / Ambulance}	¬ClearPath(Ambulance)
R5	R4 with C1	{x / Ambulance}	¬Vehicle(Ambulance) ∨ ¬EmergencyType(Ambulance)
R6	R5 with C5	{ }	¬EmergencyType(Ambulance)
R7	R6 with C6	{ }	□ (empty clause)

The empty clause □ is derived, so KB ∧ ¬Proceed(CarA) is unsatisfiable.

Therefore Proceed(CarA) is TRUE. (Reasoning chain: Ambulance is an emergency vehicle → ClearPath(Ambulance) → GreenSignal(Ambulance) → CarA is behind it → Proceed(CarA).)

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Answer:

Analysis: The present KB is NOT sufficient for two simultaneous emergency vehicles, for these reasons:

1. No conflict resolution: if Ambulance1 and Ambulance2 on crossing routes both get ClearPath, Rule 2 gives GreenSignal to both at the same junction, which is unsafe. Nothing in the KB says that two conflicting green signals cannot coexist.

2. No priority ordering: the KB has no way of deciding which emergency vehicle is served first (e.g., Fire engine vs Ambulance, or based on arrival time/distance).

3. No location/route information: Rule 1 grants a clear path to every emergency vehicle everywhere, with no link to junctions or lanes, so it cannot tell whether two paths really conflict.

4. No time dimension: signals cannot be sequenced (first serve one, then the other).

5. Rule 3 lets any vehicle behind a green-signalled vehicle proceed, even into a conflicting flow.

Additional rules/facts needed:

a) Facts: At(x, Junction), Route(x, Lane), Conflict(Lane1, Lane2), Priority(x, n) (or HigherPriority(x,y)), ArrivalTime(x, t).

b) Safety rule: ∀x ∀y: GreenSignal(x) ∧ GreenSignal(y) ∧ x ≠ y ∧ SameJunction(x,y) ∧ Conflict(x,y) → ⊥ (mutual exclusion).

c) Priority rule: ∀x ∀y: EmergencyType(x) ∧ EmergencyType(y) ∧ Conflict(x,y) ∧ HigherPriority(x,y) → ClearPath(x) ∧ Wait(y).

d) Tie-break rule: if priorities are equal, the vehicle with the earlier ArrivalTime (or nearer to the junction) is served first.

e) Wait rule: ∀y: Wait(y) → ¬GreenSignal(y) ∧ RedSignal(y), and re-evaluation when the first vehicle has passed (Passed(x) → ¬Wait(y)).

Justification: With only Rule 1 and Rule 2, both vehicles satisfy the premise and the KB entails GreenSignal(A1) ∧ GreenSignal(A2) for conflicting lanes. The safety constraint (b) would then make the KB inconsistent, which shows that the original rules over-generate. The added priority and wait rules (c)–(e) restore consistency by making ClearPath conditional, so exactly one conflicting vehicle gets the green signal at a time, while non-conflicting emergency vehicles can still be cleared simultaneously.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

Answer:

Convert each sentence (Step 1: eliminate →, Step 2: move ¬ inward, Step 3: distribute ∨ over ∧). Let S = SoilDry, I = IrrigationNeeded, W = CropWheat, D = ApplyDripMethod, K = CropAtRisk.

P1: S → I

Step 1 (eliminate →): ¬S ∨ I

Step 2 (¬ inward): no change needed. Step 3 (distribute): already a clause.

CNF: (¬S ∨ I)

P2: I ∧ W → D

Step 1: ¬(I ∧ W) ∨ D

Step 2 (De Morgan): (¬I ∨ ¬W) ∨ D

Step 3: already a single disjunction.

CNF: (¬I ∨ ¬W ∨ D)

P3: ¬D ∧ W → K

Step 1: ¬(¬D ∧ W) ∨ K

Step 2 (De Morgan, double negation): (D ∨ ¬W) ∨ K

Step 3: already a single disjunction.

CNF: (D ∨ ¬W ∨ K)

Facts: S and W are already literals, so each is a unit clause: (S), (W).

Final CNF knowledge base:

Clause	Literals	Source
1	¬S ∨ I	P1
2	¬I ∨ ¬W ∨ D	P2
3	D ∨ ¬W ∨ K	P3

4	S	Fact
5	W	Fact

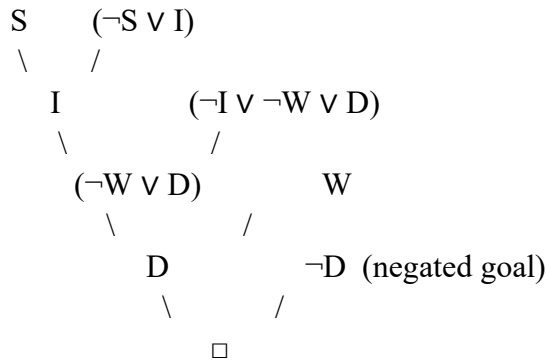
(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

Answer:

Theorem to prove: D (ApplyDripMethod). Negate the goal and add it to the KB: Clause 6: ¬D.

Step	**Resolve clauses**	**Resolvent**
7	Clause 4 (S) and Clause 1 (¬S ∨ I)	I
8	Clause 7 (I) and Clause 2 (¬I ∨ ¬W ∨ D)	¬W ∨ D
9	Clause 8 (¬W ∨ D) and Clause 5 (W)	D
10	Clause 9 (D) and Clause 6 (¬D)	□ (empty clause)

Resolution proof tree:



The empty clause □ is derived, so the KB together with ¬D is contradictory.

Hence the theorem ApplyDripMethod is proved (KB ⊨ ApplyDripMethod).

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Answer:

If SoilDry is removed, the KB is: { ¬S ∨ I, ¬I ∨ ¬W ∨ D, D ∨ ¬W ∨ K, W }.

Step-by-step effect:

1. Modus Ponens on P1 needs S. Without S, P1 (S → I) cannot fire, so IrrigationNeeded (I) is no longer derivable. (Note: ¬S is NOT derived either; the KB is only silent about S.)

2. P2 needs both I and W. With I not derivable, ApplyDripMethod (D) is no longer derivable. Resolution refutation now fails: the clause ¬D cannot be resolved down to □.

3. P3 is ¬D ∧ W → K. We know W is true, but ¬D is not established either; D is simply unknown (not proved false). Under the open-world assumption of classical logic we cannot conclude ¬D, so the premise of P3 is not satisfied.

4. Therefore K (CropAtRisk) does not follow.

Check by resolution: negate the goal K, adding ¬K. Resolving ¬K with (D ∨ ¬W ∨ K) gives (D ∨ ¬W); with W gives D; but D cannot be refuted using any remaining clause, because there is no way to get I (no S). No empty clause is derived. A model exists: S = F, I = F, W = T, D = T, K = F satisfies every clause and makes K false, so K is not entailed.

Reasoning patterns used: Modus Ponens (cannot fire without its premise), Resolution (fails to reach □), and the counter-model method to show non-entailment. Classical logic is monotonic and open-world: removing a fact can only reduce what is entailed, and an unknown fact is not treated as false. (Only under a closed-world / negation-as-failure assumption would ¬D hold and CropAtRisk be concluded, which is not valid in classical propositional logic.)

Conclusion: CropAtRisk does NOT follow from the reduced KB; the earlier conclusions IrrigationNeeded and ApplyDripMethod are also lost.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

Answer:

Belief state from percepts (Stench, Breeze, Glitter are True at the given cells):

Percept fact	**Value**
Stench[1,2]	True
Breeze[1,1]	True
Glitter[2,2]	True
Agent has visited [1,1] and [1,2] and survived	So Wumpus[1,1], Pit[1,1], Wumpus[1,2], Pit[1,2] are False

Applying Modus Ponens:

MP1: Stench[1,2] and Stench[1,2] → Wumpus adjacent to [1,2] ⟹ **Wumpus is in [1,3], [2,2] or [1,1]**, i.e., a neighbour of [1,2] (neighbours: [1,1], [1,3], [2,2]).

MP2: Breeze[1,1] and Breeze[1,1] → Pit adjacent to [1,1] ⟹ **a Pit is in [1,2] or [2,1]**, i.e., a neighbour of [1,1].

MP3: Glitter[2,2] and Glitter[x,y] → Gold at [x,y] with {x/2, y/2} ⟹ **Gold is at [2,2]**.

MP4: Since the agent is at [1,1] and [1,2] alive: ¬Wumpus[1,1] ∧ ¬Pit[1,1] → Safe[1,1]; ¬Wumpus[1,2] ∧ ¬Pit[1,2] → Safe[1,2] ⟹ **Safe[1,1] and Safe[1,2]**.

List of derived conclusions:

(i) Wumpus is adjacent to [1,2] (in [1,3] or [2,2], since [1,1] is safe);

(ii) A Pit is adjacent to [1,1] (in [2,1], since [1,2] is safe);

(iii) Gold is at [2,2];

(iv) [1,1] and [1,2] are safe.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

Answer:

Forward chaining starts from the percept facts and repeatedly fires rules until no new fact appears.

Step	**Facts / rule used**	**New knowledge**
1	Start: Stench[1,2], Breeze[1,1], Glitter[2,2]; visited and alive at [1,1], [1,2]	Safe[1,1], Safe[1,2] (by the Safe rule)
2	Rule: Stench[1,2] → Wumpus adjacent to [1,2]	Wumpus ∈ {[1,1], [1,3], [2,2]}
3	Eliminate [1,1]: Safe[1,1] (¬Wumpus[1,1])	Wumpus ∈ {[1,3], [2,2]}
4	Rule: Breeze[1,1] → Pit adjacent to [1,1]	Pit ∈ {[1,2], [2,1]}
5	Eliminate [1,2]: Safe[1,2] (¬Pit[1,2])	Pit is at [2,1]
6	Rule: Glitter[2,2] → Gold at [2,2]	Gold[2,2]

7	No further rule fires	Fixed point reached

Result:

Pit location: [2,1] (definite, since [1,2] is safe).

Wumpus location: [1,3] or [2,2] (possible locations; the percepts alone cannot decide between them).

Note: Glitter at [2,2] only shows that the gold is there. Nothing in the percepts proves ¬Wumpus[2,2], so a Wumpus in [2,2] remains logically possible.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

Answer:

Path: [1,1] → [1,2] → [2,2].

Step [1,1]: the agent is already there and alive. Safe[1,1] is proved. ✓

Step [1,2]: ¬Wumpus[1,2] ∧ ¬Pit[1,2] → Safe[1,2]. The agent has been to [1,2] and survived, and the Pit must be at [2,1] (because Breeze[1,1] requires a Pit in [1,2] or [2,1]). So [1,2] is safe. ✓

Step [2,2]: Gold is at [2,2] (Glitter), but safety is not proved. The Stench at [1,2] puts the Wumpus in [1,3] or [2,2]. We have no fact giving ¬Wumpus[2,2], so Safe[2,2] cannot be derived by the rule ¬Wumpus ∧ ¬Pit → Safe. The Pit risk at [2,2] is low (the Pit is at [2,1]), but the Wumpus risk is real. ✗ (not proved)

Evaluation: the first two cells are safe, but the last cell [2,2] is not proved safe: it is one of the two possible Wumpus locations. So the full path is NOT guaranteed safe.

Recommendation: the path is safe up to [1,2]. Before entering [2,2] the agent should gather more evidence (for example, shoot an arrow into [2,2] from [1,2], or find a percept that places the Wumpus in [1,3]). Once ¬Wumpus[2,2] is derived, Safe[2,2] follows by the Safe rule and the agent can collect the gold.

Conclusion: the path [1,1] → [1,2] → [2,2] is safe up to [1,2]; entering [2,2] to grab the gold is not yet logically proved safe.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand